User Manual

for S32 THERMAL Driver

Document Number: UM11THERMALASR4.4 Rev0000R4.0.2 Rev. 1.0

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

This User Manual describes NXP Semiconductor Thermal for S32. Thermal driver configuration parameters and deviations from the specification are described in Driver chapter of this document. Thermal driver requirements and APIs are described in the Thermal driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
C/CPP	C and C++ Source Code
DET	Development Error Tracer
ECU	Electronic Control Unit
N/A	Not Applicable
RAM	Random Access Memory
SWS	Software Specification

2.5 Reference List

#	Title	Version
1	S32G2 Reference Manual	Rev. 7, February 2023
2	S32G3 Reference Manual	Rev. 3, Feb 2023
3	S32R45 Reference Manual	Rev. 3, 12/2021
4	S32G2 Data Sheet	Rev. 6, Dec 2022
5	S32G3 Data Sheet	Rev. 2, RC, Feb 2023
6	S32R45 Data Sheet	Rev. 3, RC, 11/2022
7	VR5510 Data Sheet	Rev. 5, April 2022
8	S32G2 Errata for Mask 0P77B	v3.0
9	S32G2 Errata for Mask 1P77B	v1.0
10	S32G3 Errata for Mask 1P72B	Rev. 2.1
11	S32R45 Errata for Mask P57D	Rev. 2.0

Chapter 3

Driver

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See Table Reference List).

For CDD: Thermal is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module.

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The Thermal driver initializes and controls the Thermal Monitoring Unit hardware. It doesn't communicate directly with the hardware; rather, it forwards calls to a lower-level driver named "TMU IP".

The Thermal driver provides services for:

- initializing the hardware using an application-provided configuration structure;
- putting the hardware peripheral in sleep mode and waking it up again;
- enabling and disabling monitoring sites (there are three temperature sensors placed at three sites inside the microcontroller);
- enabling and disabling thresholds that generate interrupts;
- retrieving the current temperature at each site, or historical minimums or maximums.

3.3 Hardware Resources

The S32 devices contain one TMU peripheral. For more details, please consult HW reference manual.

3.4 Deviations from Requirements

None.

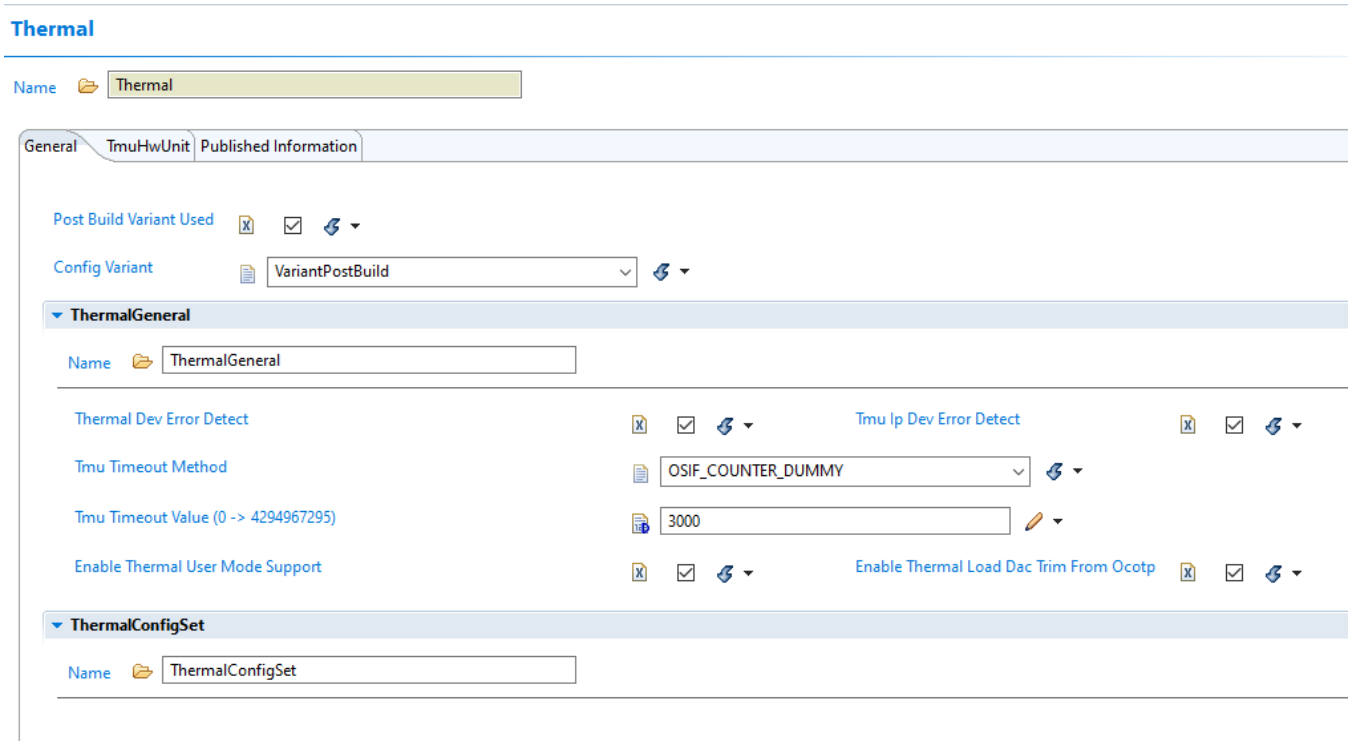
3.5 Driver Limitations

None.

3.6 Driver usage and configuration tips

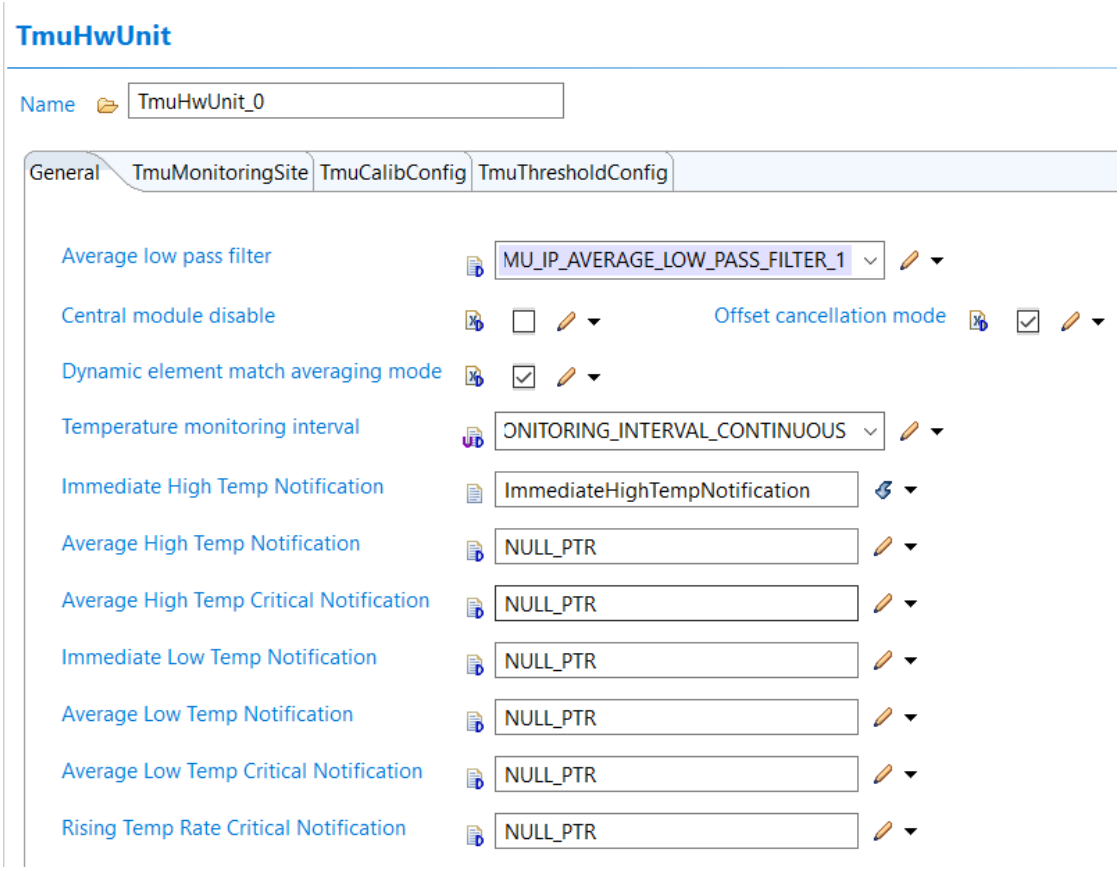
A typical usecase for Thermal driver would imply configuring temperature monitoring site, temperature low pass filter used for computing average temperature, temperature thresholds, and alarm notifications. Thermal_Init() function configures the sensor calibration values for the temperature sensor.

In Tresos configuration plug-in for the Thermal Driver there are three main containers, from which two contain parameters that are configurable (General and TmuHwUnit), and one which contains common information regarding the module - read-only parameters (Published information).



General tab contains Config Time Support, enable / disable parameters for development error detection, timeout method and value drop-downs, and enable / disable parameter for user mode.

TmuHwUnit has four containers: General, TmuMonitoringSite, TmuCalibConfig, and TmuThresholdConfig. General contains configurable fields for Temperature Monitoring interval, average low pass filter for calculating the average temperature, etc.



TmuCalibConfig could (is not mandatory) contain calibration points configurations. This container allows configuring the calibration configuration points. If the list from TmuHwUnit/TmuCalibConfig container in Tresos is empty (resulting in pointer to calibration config NULL), the driver will use the default calibration points specified by peripheral hardware documentation.

Note

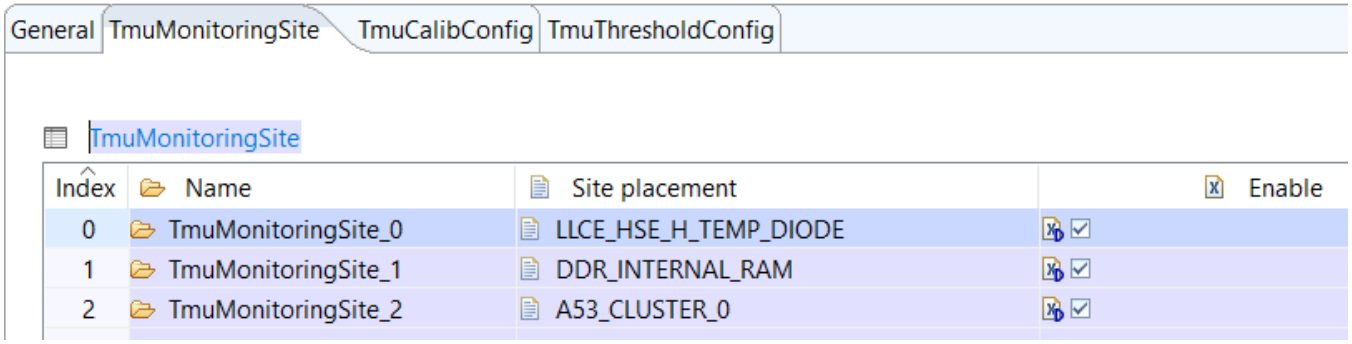
IMPORTANT NOTE:

- For default calibration values, the array from TmuHwUnit/TmuCalibConfig container in Tresos must be empty. Configure TmuHwUnit/TmuCalibConfig container ONLY if application requires custom calibration points, to override the default calibration points.
- The user can select to include the Trim values from fuses to the calibration points by the option Thermal↩ General/ThermalLoadDacTrimFromOcotp

TmuThresholdConfig contains thresholds configurations. TmuMonitoringSite contains configured monitoring sites. In this container there must be at least one monitoring site configured and enabled. Maximum number of sites, as well as their physical placement depend on the current platform.

Note

If no monitoring site is enabled, an error will be raised in the Configurator.



3.7 Runtime errors

The driver generates the following DET errors at runtime.

Function	Error Code	Condition triggering the error
Thermal_Init	THERMAL_E_ALREADY_INITIALIZED	Calling Thermal_Init twice in a row
Thermal_Init	THERMAL_E_INIT_FAILED	Thermal module is not initialized successfully
Thermal_Deinit	THERMAL_E_NOT_INITIALIZED	Thermal_Init() was not called before
Thermal_EnableMonitoringSite	THERMAL_E_NOT_INITIALIZED	Thermal_Init() was not called before
Thermal_EnableMonitoringSite	THERMAL_E_SITE_MASK_NOT_VALID	Invalid parameter
Thermal_EnableMonitoringSite	THERMAL_E_NEED_LOW_POWER_MODE	Function can only be called in low-power mode (see Thermal_SetPowerMode)
Thermal_DisableMonitoringSite	THERMAL_E_NOT_INITIALIZED	Thermal_Init() was not called before
Thermal_DisableMonitoringSite	THERMAL_E_SITE_MASK_NOT_VALID	Invalid parameter
Thermal_DisableMonitoringSite	THERMAL_E_NEED_LOW_POWER_MODE	Function can only be called in low-power mode (see Thermal_SetPowerMode)
Thermal_SetPowerMode	THERMAL_E_NOT_INITIALIZED	Thermal_Init() was not called before
Thermal_SetPowerMode	THERMAL_E_NEED_LOW_POWER_MODE	Function called to set monitoring (normal power) mode, but module not currently in low power.
Thermal_SetPowerMode	THERMAL_E_NEED_MONITORING_MODE	Function called to set low power mode, but module not currently in monitoring mode.

Function	Error Code	Condition triggering the error
Thermal_GetTempMinMaxLevel	THERMAL_E_NOT_INITIALIZED	Thermal_Init() was not called before
Thermal_GetTempMinMaxLevel	THERMAL_E_NEED_MONITORING_MODE	Function called while not in monitoring mode
Thermal_GetTempMaxRate	THERMAL_E_NOT_INITIALIZED	Thermal_Init() was not called before
Thermal_GetTempMaxRate	THERMAL_E_NEED_MONITORING_MODE	Function called while not in monitoring mode
Thermal_GetTemp	THERMAL_E_NOT_INITIALIZED	Thermal_Init() was not called before
Thermal_GetTemp	THERMAL_E_NEED_MONITORING_MODE	Function called while not in monitoring mode
Thermal_GetTemp	THERMAL_E_SITE_INDEX_TOO_LARGE	Invalid parameter
Thermal_SetTempThreshold	THERMAL_E_NOT_INITIALIZED	Thermal_Init() was not called before
Thermal_SetTempThreshold	THERMAL_E_NEED_MONITORING_MODE	Function called while not in monitoring mode
Thermal_EnableNotifications	THERMAL_E_NOT_INITIALIZED	Thermal_Init() was not called before
Thermal_EnableNotifications	THERMAL_E_NOTIF_NOT_CONFIGURED	Function called for a notification for which no handler was configured
Thermal_DisableNotifications	THERMAL_E_NOT_INITIALIZED	Thermal_Init() was not called before
Thermal_SetMeasurementInterval	THERMAL_E_NOT_INITIALIZED	Thermal_Init() was not called before
Thermal_SetMeasurementInterval	THERMAL_E_NEED_LOW_POWER_MODE	Function called in monitoring mode, but may be called only in low-power mode.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

#define <Mip>Conf_<Container_ShortName>_<Container_ID>

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module Thermal
  - Container ThermalGeneral
    - Parameter ThermalDevErrorDetect
    - Parameter TmuIpDevErrorDetect
    - Parameter TmuTimeoutMethod
    - Parameter TmuTimeoutVal
    - Parameter ThermalEnableUserModeSupport
    - Parameter ThermalLoadDacTrimFromOcotp
  - Container ThermalConfigSet
    - Container TmuHwUnit
      - Parameter ThermalAverageLowPassFilterType
      - Parameter ThermalCentralModuleDisable
      - Parameter ThermalOffsetCancellation
      - Parameter ThermalDynamicMatchAvg
      - Parameter ThermalMeasurementInterval
      - Parameter ThermalImmHighNotif
      - Parameter ThermalAvgHighNotif
      - Parameter ThermalAvgHighCriticalNotif
      - Parameter ThermalImmLowNotif
      - Parameter ThermalAvgLowNotif
      - Parameter ThermalAvgLowCriticalNotif
      - Parameter ThermalRisingRateNotif
      - Parameter ThermalFallingRateNotif
      - Container TmuMonitoringSite
      - Parameter TmuSitePlacement
      - Parameter TmuEnableSite
      - Container TmuCalibConfig
      - Parameter ThermalSensorValue
      - Parameter ThermalTempRange
      - Container TmuThresholdConfig

- Parameter ThermalThresholdType
- Parameter ThermalThresholdValue
- Parameter ThermalThrEnable
- Parameter ThermalInterruptEnable

– Container CommonPublishedInformation

* Parameter ArReleaseMajorVersion
* Parameter ArReleaseMinorVersion
* Parameter ArReleaseRevisionVersion
* Parameter ModuleId
* Parameter SwMajorVersion
* Parameter SwMinorVersion
* Parameter SwPatchVersion
* Parameter VendorApiInfix
* Parameter VendorId

4.1 Module Thermal

Thermal config set

Included containers:

- ThermalGeneral
- ThermalConfigSet
- CommonPublishedInformation

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container ThermalGeneral

General configuration (parameters) of the Thermal Driver software module.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter ThermalDevErrorDetect

Enables/disables development error detection for Thermal module

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.4 Parameter TmuIpDevErrorDetect

Enables/disables development error detection for Tmu Ip

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.5 Parameter TmuTimeoutMethod

Configures the timeout method. Based on this selection, a certain timeout method from OsIf will be used in the driver.

Note: If OSIF_COUNTER_SYSTEM or OSIF_COUNTER_CUSTOM are selected make sure the corresponding timer is enabled in OsIf General configuration.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COU↩ NTER_CUSTOM']

4.6 Parameter TmuTimeoutVal

Tmu timeout value

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	3000
max	4294967295
min	0

4.7 Parameter ThermalEnableUserModeSupport

When this parameter is enabled, the Thermal module will adapt to run from User Mode, by configuring REG_PROT for Thermal IPs

Note: The Thermal driver code can be executed at any time from both supervisor and user mode.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.8 Parameter ThermalLoadDacTrimFromOcotp

When this parameter is enabled, the module will load DAC_TRIM value from Ocotp and add it to default Sensor values.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.9 Container ThermalConfigSet

This is the base container that contains the post-build selectable configuration parameters

Included subcontainers:

- TmuHwUnit

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.10 Container TmuHwUnit

TMU Configuration Array

Included subcontainers:

- TmuMonitoringSite
- TmuCalibConfig
- TmuThresholdConfig

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	2
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.11 Parameter ThermalAverageLowPassFilterType

This is used to set the average low pass filter. The average temperature is calculated using this formula: ALPF x Current_Temp + (1 - ALPF) x Average_Temp.

Note: TMU_IP_AVERAGE_LOW_PASS_FILTER_0_xx means ALPF = 0.xx. For example, TMU_IP_AVERAGE_LOW_PASS_FILTER_0_25 means ALPF = 0.25.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	TMU_IP_AVERAGE_LOW_PASS_FILTER_1
literals	['TMU_IP_AVERAGE_LOW_PASS_FILTER_1', 'TMU_IP_AVERAGE↩ _LOW_PASS_FILTER_0_5', 'TMU_IP_AVERAGE_LOW_PASS_FILT↩ ER_0_25', 'TMU_IP_AVERAGE_LOW_PASS_FILTER_0_125']

4.12 Parameter ThermalCentralModuleDisable

The configuration of central module mode, false/true - enable/disable

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.13 Parameter ThermalOffsetCancellation

The configuration of offset cancellation mode, false/true - disable/enable

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.14 Parameter ThermalDynamicMatchAvg

The configuration of dynamic averaging mode, false/true - disable/enable

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.15 Parameter ThermalMeasurementInterval

This enum is used to set temperature monitoring interval in seconds. The time interval depends on TMU clock frequency

and is determined using this formula: [2 ^ (23 + TMI)] x TMU clock period in seconds, where TMI is Temperature Monitoring Interval

TMU_IP_TEMP_MONITORING_INTERVAL_CONTINUOUS -> Disable temperature monitoring interval in seconds, monitoring is continuous

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	TMU_IP_TEMP_MONITORING_INTERVAL_CONTINUOUS
literals	['TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_0', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_1', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_2', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_3', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_4', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_5', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_6', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_7', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_8', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_9', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_10', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_11', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_12', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_13', 'TMU_IP_TEMP_MONITORING_INTERVAL_SECOND_SELECT_14', 'TMU_IP_TEMP_MONITORING_INTERVAL_CONTINUOUS']

4.16 Parameter ThermalImmHighNotif

This function pointer is called whenever the high temperature immediate threshold issues an interrupt.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.17 Parameter ThermalAvgHighNotif

This function pointer is called whenever the high temperature average threshold issues an interrupt.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.18 Parameter ThermalAvgHighCriticalNotif

This function pointer is called whenever the high temperature average critical threshold issues an interrupt.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.19 Parameter ThermalImmLowNotif

This function pointer is called whenever the low temperature immediate threshold issues an interrupt.

Property	Value
type	ECUC-FUNCTION-NAME-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.20 Parameter ThermalAvgLowNotif

This function pointer is called whenever the low temperature average threshold issues an interrupt.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.21 Parameter ThermalAvgLowCriticalNotif

This function pointer is called whenever the low temperature average critical threshold issues an interrupt.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.22 Parameter ThermalRisingRateNotif

This function pointer is called whenever the The rising temperature difference between two measurements issues an interrupt.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.23 Parameter ThermalFallingRateNotif

This function pointer is called whenever the falling temperature difference between two measurements issues an interrupt.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.24 Container TmuMonitoringSite

Monitoring sites array

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	3
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.25 Parameter TmuSitePlacement

This parameter defines the physical placement for the Tmu sensor. For example, DDR_INTERNAL_RAM means the sensor is placed between the DDR and the Internal RAM. A53_CORE means the sensor is near A53 Core.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	DDR_INTERNAL_RAM
literals	['DDR_INTERNAL_RAM', 'A53_CLUSTER_0', 'LLCE_HSE_H_TEMP_↩ DIODE']

4.26 Parameter TmuEnableSite

Enable monitoring site.

Depending on which sites are enabled, the site mask will be computed.Zero or more TMU_IP_MONITORING_SITE_ values, OR-ed together.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.27 Container TmuCalibConfig

This container allows configuring the calibration configuration points.

Driver Init function configures the sensor calibration values for the temperature sensor.

If this list is empty (resulting in pointer to calibration config NULL_PTR), the driver will use the default calibration points specified by peripheral hardware documentation.

IMPORTANT NOTE: For default calibration values, this array must be empty. Configure this container ONLY if application requires custom calibration points, to override the default calibration points.

.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	16
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.28 Parameter ThermalSensorValue

The sensor value

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	41
max	511
min	0

4.29 Parameter ThermalTempRange

The temperature range. The value is expressed in Kelvin unit.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	233
max	433
min	233

4.30 Container TmuThresholdConfig

This array contains the Threshold configuration (parameters).

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	8
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.31 Parameter ThermalThresholdType

Type of threshold value

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	TMU_IP_THRESHOLD_LOW_TEMP_AVERAGE
literals	['TMU_IP_THRESHOLD_HIGH_TEMP_IMMEDIATE', 'TMU_IP_THR↩ ESHOLD_HIGH_TEMP_AVERAGE', 'TMU_IP_THRESHOLD_HIGH_T↩ EMP_AVERAGE_CRITICAL', 'TMU_IP_THRESHOLD_LOW_TEMP_I↩ MMEDIATE', 'TMU_IP_THRESHOLD_LOW_TEMP_AVERAGE', 'TMU↩ _IP_THRESHOLD_LOW_TEMP_AVERAGE_CRITICAL', 'TMU_IP_T↩ HRESHOLD_RISING_TEMP_RATE_CRITICAL', 'TMU_IP_THRESHO↩ LD_FALLING_TEMP_RATE_CRITICAL']

4.32 Parameter ThermalThresholdValue

Temperature threshold value. The value is expressed in Kelvin unit

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	233
max	433
min	0

4.33 Parameter ThermalThrEnable

Enable or disable threshold

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.34 Parameter ThermalInterruptEnable

Interrupt enable configuration

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.35 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.36 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION

Property	Value
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.37 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.38 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0

Property	Value
max	0
min	0

4.39 Parameter ModuleId

Module ID of this module from Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.40 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.41 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.42 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2
max	2
min	2

4.43 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.44 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Chapter 6

Module Documentation

6.1 Thermal_driver

6.1.1 Detailed Description

Macros

- #define THERMAL_INVALID_TEMP

  Value that represents an invalid temperature. This value is returned by functions in case a measurement is not yet completed in hardware.
- #define THERMAL_INVALID_TEMP_RATE

  Value that represents an invalid temperature rate. This value is returned by functions in case this information is not yet available in hardware.
- #define THERMAL_E_NEED_LOW_POWER_MODE
- #define THERMAL_INIT_ID
- #define THERMAL_MONITORING_SITE_0

  Monitoring sites. Can be OR-ed together.

Types Reference

- typedef uint16 Thermal_TempValueType

  Data type used for holding a temperature value. One unit represents half a degree Kelvin, starting from zero Kelvin.
- typedef uint8 Thermal_TempRateValueType

  Data type used for holding a temperature rise or fall rate. One unit represents half a degree Kelvin.
- typedef Tmu_Ip_ConfigType Thermal_ConfigType

  Structure that holds configuring information required for initializing the peripheral.
- typedef uint8 Thermal_MonitoringSiteType

  Bitmask value that specifies which measuring sites should be affected by certain operations. Bit 0 represents the first site, bit 1 the second site, bit 2 the third site.

Enum Reference

- enum Thermal_MonitoringIntervalType

  Temperature monitoring interval setting.
- enum Thermal_ThresholdType

  Temperature threshold type.
- enum Thermal_PowerModeType

  Working mode of the Thermal peripheral.
- enum Thermal_TempLevelType

  Enum that specifies either a low or a high temperature setting.
- enum Thermal_TempRateType

  Enum that specifies either a rising or a falling temperature.
- enum Thermal_ReportType

  Enum that specifies either an immediate or an average temperature.

Function Reference

- void Thermal_Init (const Thermal_ConfigType *const ConfigPtr)

  This function initializes the Thermal CDD.
- void Thermal_Deinit (void)

  Un-initializes the Thermal CDD by undoing the work done by Thermal_Init().
- void Thermal_EnableMonitoringSite (Thermal_MonitoringSiteType SiteMask)

  Enables monitoring on specified sites.
- void Thermal_DisableMonitoringSite (Thermal_MonitoringSiteType SiteMask)

  Disables monitoring on specified sites.
- void Thermal_SetPowerMode (Thermal_PowerModeType Mode)

  Sets the peripheral's power mode.
- Thermal_TempValueType Thermal_GetTempMinMaxLevel (Thermal_TempLevelType TempLevelType, boolean ClearValidFlag)

  Retrieves the maximum or minimum temperature recorded, and optionally clears this information in hardware.
- Thermal_TempRateValueType Thermal_GetTempMaxRate (Thermal_TempRateType TempRateType, boolean ClearValidFlag)

  Retrieves the maximum or minimum temperature rate recorded, and optionally clears this information in hardware.
- Thermal_TempValueType Thermal_GetTemp (uint8 SiteIndex, Thermal_ReportType ReportType)

  Retrieves the current temperature at a single site.
- void Thermal_SetTempThreshold (Thermal_ThresholdType ThresholdType, uint16 ThresholdValue)

  Sets the threshold temperature for a given type of threshold.
- void Thermal_EnableNotifications (uint32 NotificationsMask)

  Enables selected notifications.
- void Thermal_DisableNotifications (uint32 NotificationsMask)

  Disables selected notifications.
- void Thermal_SetMeasurementInterval (Thermal_MonitoringIntervalType MonitoringInterval)

  Sets measurement interval. Can be called only while in idle power mode.

6.1.2 Macro Definition Documentation

6.1.2.1 THERMAL_INVALID_TEMP

```
#define THERMAL_INVALID_TEMP
```

Value that represents an invalid temperature. This value is returned by functions in case a measurement is not yet completed in hardware.

Definition at line 131 of file CDD_Thermal.h.

6.1.2.2 THERMAL_INVALID_TEMP_RATE

```
#define THERMAL_INVALID_TEMP_RATE
```

Value that represents an invalid temperature rate. This value is returned by functions in case this information is not yet available in hardware.

Definition at line 136 of file CDD_Thermal.h.

6.1.2.3 THERMAL_E_NEED_LOW_POWER_MODE

```
#define THERMAL_E_NEED_LOW_POWER_MODE
```

Various error codes that this Thermal driver uses for generating Det errors.

Definition at line 139 of file CDD_Thermal.h.

6.1.2.4 THERMAL_INIT_ID

```
#define THERMAL_INIT_ID
```

Various service IDs that this Thermal driver uses for generating Det errors.

Definition at line 149 of file CDD_Thermal.h.

6.1.2.5 THERMAL_MONITORING_SITE_0

```
#define THERMAL_MONITORING_SITE_0
```

Monitoring sites. Can be OR-ed together.

Definition at line 109 of file CDD_Thermal_Types.h.

6.1.3 Types Reference

6.1.3.1 Thermal_TempValueType

```
typedef uint16 Thermal_TempValueType
```

Data type used for holding a temperature value. One unit represents half a degree Kelvin, starting from zero Kelvin.

Definition at line 218 of file CDD_Thermal_Types.h.

6.1.3.2 Thermal_TempRateValueType

```
typedef uint8 Thermal_TempRateValueType
```

Data type used for holding a temperature rise or fall rate. One unit represents half a degree Kelvin.

Definition at line 225 of file CDD_Thermal_Types.h.

6.1.3.3 Thermal_ConfigType

```
typedef Tmu_Ip_ConfigType Thermal_ConfigType
```

Structure that holds configuring information required for initializing the peripheral.

Definition at line 231 of file CDD_Thermal_Types.h.

6.1.3.4 Thermal_MonitoringSiteType

```
typedef uint8 Thermal_MonitoringSiteType
```

Bitmask value that specifies which measuring sites should be affected by certain operations. Bit 0 represents the first site, bit 1 the second site, bit 2 the third site.

Definition at line 238 of file CDD_Thermal_Types.h.

6.1.4 Enum Reference

6.1.4.1 Thermal_MonitoringIntervalType

```
enum Thermal_MonitoringIntervalType
```

Temperature monitoring interval setting.

This enum is used to set temperature monitoring interval in seconds. The time interval depends on TMU clock frequency and is determined using this formula: [2 ^ (23 + TMI)] x TMU clock period in seconds, where TMI is Temperature Monitoring Interval

Enumerator

THERMAL_MONITORING_INTERVAL_TYPE_CONTINUOUS	Disable temperature monitoring interval in seconds, monitoring is continuous

Definition at line 127 of file CDD_Thermal_Types.h.

6.1.4.2 Thermal_ThresholdType

enum Thermal_ThresholdType

Temperature threshold type.

Definition at line 151 of file CDD_Thermal_Types.h.

6.1.4.3 Thermal_PowerModeType

enum Thermal_PowerModeType

Working mode of the Thermal peripheral.

Definition at line 167 of file CDD_Thermal_Types.h.

6.1.4.4 Thermal_TempLevelType

enum Thermal_TempLevelType

Enum that specifies either a low or a high temperature setting.

Used by functions such as Thermal_GetTempMinMaxLevel.

Definition at line 179 of file CDD_Thermal_Types.h.

6.1.4.5 Thermal_TempRateType

enum Thermal_TempRateType

Enum that specifies either a rising or a falling temperature.

Used by functions such as Thermal_GetTempMaxRate.

Definition at line 191 of file CDD_Thermal_Types.h.

6.1.4.6 Thermal_ReportType

```
enum Thermal_ReportType
```

Enum that specifies either an immediate or an average temperature.

Used by functions such as Thermal_GetTemp.

Definition at line 203 of file CDD_Thermal_Types.h.

6.1.5 Function Reference

6.1.5.1 Thermal_Init()

```
void Thermal_Init (
            const Thermal_ConfigType *const ConfigPtr )
```

This function initializes the Thermal CDD.

Parameters

in	*ConfigPtr*	Pointer to the configuration structure.

6.1.5.2 Thermal_Deinit()

```
void Thermal_Deinit (
            void  )
```

Un-initializes the Thermal CDD by undoing the work done by Thermal_Init().

6.1.5.3 Thermal_EnableMonitoringSite()

```
void Thermal_EnableMonitoringSite (
            Thermal_MonitoringSiteType SiteMask )
```

Enables monitoring on specified sites.

Parameters

in	*SiteMask*	A combination of THERMAL_MONITORING_SITE_ values

6.1.5.4 Thermal_DisableMonitoringSite()

```
void Thermal_DisableMonitoringSite (
            Thermal_MonitoringSiteType SiteMask )
```

Disables monitoring on specified sites.

Parameters

in	*SiteMask*	A combination of THERMAL_MONITORING_SITE_ values

6.1.5.5 Thermal_SetPowerMode()

```
void Thermal_SetPowerMode (
            Thermal_PowerModeType Mode )
```

Sets the peripheral's power mode.

Parameters

in	*Mode*	one of THERMAL_POWERMODE_NORMAL / THERMAL_POWERMODE_LOWPOWER.

6.1.5.6 Thermal_GetTempMinMaxLevel()

```
Thermal_TempValueType Thermal_GetTempMinMaxLevel (
            Thermal_TempLevelType TempLevelType,
            boolean ClearValidFlag )
```

Retrieves the maximum or minimum temperature recorded, and optionally clears this information in hardware.

Parameters

in	*TempLevelType*	specifies whether to retrieve the maximum or minimum.
in	*ClearValidFlag*	TRUE to clear the recorded temperature.

Returns

Thermal_TempValueType: Tmperature value

6.1.5.7 Thermal_GetTempMaxRate()

```
Thermal_TempRateValueType Thermal_GetTempMaxRate (
            Thermal_TempRateType TempRateType,
            boolean ClearValidFlag )
```

Retrieves the maximum or minimum temperature rate recorded, and optionally clears this information in hardware.

Parameters

in	*TempRateType*	specifies whether to retrieve the rising or the falling rate.
in	*ClearValidFlag*	TRUE to clear the recorded temperature.

Returns

Thermal_TempRateValueType: Temperature rate value

6.1.5.8 Thermal_GetTemp()

```
Thermal_TempValueType Thermal_GetTemp (
            uint8 SiteIndex,
            Thermal_ReportType ReportType )
```

Retrieves the current temperature at a single site.

Parameters

in	*SiteIndex*	0 for the first site, 1 for the second site, 2 for the third site.
in	*ReportType*	specifies whether to retrieve the immediate or the average temperature.

Returns

temperature value.

Thermal_TempValueType: Temperature value

6.1.5.9 Thermal_SetTempThreshold()

```
void Thermal_SetTempThreshold (
            Thermal_ThresholdType ThresholdType,
            uint16 ThresholdValue )
```

Sets the threshold temperature for a given type of threshold.

Parameters

in	*ThresholdType*	type of threshold.
in	*ThresholdValue*	temperature in degrees Kelvin, with each unit representing half a degree.

6.1.5.10 Thermal_EnableNotifications()

```
void Thermal_EnableNotifications (
            uint32 NotificationsMask )
```

Enables selected notifications.

Parameters

in	*NotificationsMask*	combination of TMU_IP_INTERRUPT_ values

6.1.5.11 Thermal_DisableNotifications()

```
void Thermal_DisableNotifications (
            uint32 NotificationsMask )
```

Disables selected notifications.

Parameters

in	*NotificationsMask*	combination of TMU_IP_INTERRUPT_ values

6.1.5.12 Thermal_SetMeasurementInterval()

```
void Thermal_SetMeasurementInterval (
            Thermal_MonitoringIntervalType MonitoringInterval )
```

Sets measurement interval. Can be called only while in idle power mode.

Parameters

in	*MonitoringInterval*	value. See the description of each enum value for more information.

6.2 Tmu IPL

6.2.1 Detailed Description

Data Structures

- struct Tmu_Ip_CalibrationConfigType

  Structure to configure the calibration point and temperature range control. *More...*
- struct Tmu_Ip_ThresholdConfigType

  Structure to configure the temperature threshold. *More...*
- struct Tmu_Ip_ConfigType

  TMU configuration structure. *More...*

Macros

- #define TMU_IP_INVALID_TEMPERATURE

  Ivalid temperature value. Invalid temperature and temperature rate values.
- #define TMU_IP_MONITORING_SITE_0

  Site masks. Use these masks wherever you want to select the site.
- #define TMU_IP_INTERRUPT_IMMEDIATE_HIGH_TEMP

  The temperature threshold interrupt source masks.
- #define TMU_IP_INTERRUPT_AVERAGE_HIGH_TEMP
- #define TMU_IP_INTERRUPT_AVERAGE_HIGH_TEMP_CRITICAL
- #define TMU_IP_INTERRUPT_IMMEDIATE_LOW_TEMP
- #define TMU_IP_INTERRUPT_AVERAGE_LOW_TEMP
- #define TMU_IP_INTERRUPT_AVERAGE_LOW_TEMP_CRITICAL
- #define TMU_IP_INTERRUPT_RISING_TEMP_RATE_CRITICAL
- #define TMU_IP_INTERRUPT_FALLING_TEMP_RATE_CRITICAL
- #define TMU_IP_STATUS_TIME_INTERVAL_EXCEEDED

  The monitoring status flag masks.
- #define TMU_IP_STATUS_OUT_OF_RANGE_LOW_TEMP
- #define TMU_IP_STATUS_OUT_OF_RANGE_HIGH_TEMP

Enum Reference

- enum Tmu_Ip_AverageLowPassFilterType

  Average low pass filter setting.
- enum Tmu_Ip_TempMonitoringIntervalType

  Temperature monitoring interval setting.
- enum Tmu_Ip_ReportTempType

  Report temperature settings.
- enum Tmu_Ip_ThresholdType

  Temperature threshold settings.
- enum Tmu_Ip_ModeType

  Working mode of the TMU peripheral. Values correspond to those in the MODE field of of the TMR register.
- enum Tmu_Ip_StatusType

  Defines the Tmu status values.

Initialization and De-initialization

- Tmu_Ip_StatusType Tmu_Ip_Init (const uint32 Instance, const Tmu_Ip_ConfigType *const ConfigPtr)

  This function initializes ...
- void Tmu_Ip_Deinit (const uint32 Instance)

  This function shall De-Init the TMU for all registers to default value.

Temperature monitoring

- void Tmu_Ip_StartMonitoring (const uint32 Instance)

  This function starts monitoring the temperature on sites previously selected with Tmu_Ip_Init or Tmu_Ip_Enable↩ MonitoringSite. This function should be called after TMU initialization is done.
- void Tmu_Ip_StopMonitoring (const uint32 Instance)

  This function stop monitoring the temperature on sites.
- void Tmu_Ip_EnableMonitoringSite (const uint32 Instance, const uint8 TmuSiteMask)

  This function shall enable the site which will be monitored. The site mask input value can be set using TMU_IP↩ _MONITORING_SITE_ defines.
- void Tmu_Ip_DisableMonitoringSite (const uint32 Instance, const uint8 TmuSiteMask)

  This function shall disable the selected site from monitoring. The site mask input value can be set using TMU_I↩ P_MONITORING_SITE_ defines.
- void Tmu_Ip_SetTempThreshold (const uint32 Instance, const Tmu_Ip_ThresholdType ThresholdType, const uint16 ThresholdValue)

  This function shall set the temperature threshold.
- uint16 Tmu_Ip_GetTempMinMaxLevel (const uint32 Instance, const Tmu_Ip_TempLevelType TempLevel↩ Type, const boolean ClearValidFlag)

  This function reads the minimum or maximum temperature recorded at one site. If the temperature is invalid, it will return TMU_IP_INVALID_TEMPERATURE value. The return value is Q9.1 format: 9 bits for highest temperature recorded and 1 bit for 0.5 degrees resolution. User can clear the current temperature value by setting ClearValidFlag parameter.
- uint8 Tmu_Ip_GetTempMaxRate (const uint32 Instance, const Tmu_Ip_TempRateType TempRateType, const boolean ClearValidFlag)

  Retrieves the maximum rate the temperature has risen or fallen, and clears that rate if ClearValidFlag is TRUE. If the temperature is invalid (cleared), it will return TMU_IP_INVALID_TEMPERATURE_RATE value.
- void Tmu_Ip_GetTemp (const uint32 Instance, const uint8 TmuMonitoringSite, const Tmu_Ip_ReportTempType TmuTempType, uint16 *const TmuTempVal)

  This function shall read the last temperature at the selected site If the temperature is invalid, it will return TMU_↩ IP_INVALID_TEMPERATURE value. The return value is Q9.1 format : 9 bits for highest temperature recorded and 1 bit for 0.5 degrees resolution.

Interrupt and flags

- void Tmu_Ip_EnableNotifications (const uint32 Instance, const uint32 NotificationsMask)

  This function enables the notifications for the selected types of thresholds. The interrupt mask input value can be set using TMU_IP_INTERRUPT_ defines.
- void Tmu_Ip_DisableNotifications (const uint32 Instance, const uint32 NotificationsMask)

  This function disables the notifications for the selected types of thresholds. The interrupt mask input value can be set using TMU_IP_INTERRUPT_ defines.

Measurement interval

- void Tmu_Ip_SetMeasurementInterval (const uint32 Instance, const Tmu_Ip_TempMonitoringIntervalType TmuMonitoringInterval)

  This function shall set the interval for the temperature monitoring. It should only be changed when monitoring is disabled.
- Tmu_Ip_ModeType Tmu_Ip_GetMode (const uint32 Instance)

  Retrieves the current mode. (TMU_IP_MODE_MONITORING after a call to Tmu_Ip_StartMonitoring, otherwise TMU_IP_MODE_IDLE_LOW_POWER)
- #define **THERMAL_STOP_SEC_CODE**

6.2.2 Data Structure Documentation

6.2.2.1 struct Tmu_Ip_CalibrationConfigType

Structure to configure the calibration point and temperature range control.

This is used to setup calibration point, each calibration point is related to the Temperature range control register.

Definition at line 266 of file Tmu_Ip_Types.h.

Data Fields

Type	Name	Description
uint16	SensorValue	The sensor value
uint16	TempRange	The temperature range

6.2.2.2 struct Tmu_Ip_ThresholdConfigType

Structure to configure the temperature threshold.

This structure holds the configuration settings for the threshold value of TMU.

Definition at line 278 of file Tmu_Ip_Types.h.

Data Fields

Type	Name	Description
boolean	Enable	Enable threshold
Tmu_Ip_ThresholdType	ThresholdType	Type of threshold value
uint16	ThresholdValue	Temperature threshold value
boolean	InterruptEnable	Enable interrupt

6.2.2.3 struct Tmu_Ip_ConfigType

TMU configuration structure.

Definition at line 304 of file Tmu_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	MonitoringSitesMask	Zero or more TMU_IP_MONITORING_SITE_ values, OR-ed together.
Tmu_Ip_AverageLowPassFilterType	LowPassFilter	The average low pass filter
boolean	CentralModuleDisable	The configuration of central module mode, false/true - enabled/disabled
boolean	OffsetCancellation	Configure offset cancellation mode, false/true - disable/enable
boolean	DynamicMatchAvrg	Configure dynamic element match averaging mode, false/true - disable/enable
Tmu_Ip_TempMonitoringIntervalType	MeasurementInterval	The temperature monitoring interval value
uint8	NumCalibrationConfigs	The number of calibration point
const Tmu_Ip_CalibrationConfigType *	CalibrationConfig	The calibration configuration
uint8	NumThresholds	Number of threshold configuration
const Tmu_Ip_ThresholdConfigType *	ThresholdConfig	The pointer point to threshold configuration structure
const Tmu_Ip_NotificationsType *	Notifications	

6.2.3 Macro Definition Documentation

6.2.3.1 TMU_IP_INVALID_TEMPERATURE

```
#define TMU_IP_INVALID_TEMPERATURE
```

Ivalid temperature value. Invalid temperature and temperature rate values.

Definition at line 103 of file Tmu_Ip_Types.h.

6.2.3.2 TMU_IP_MONITORING_SITE_0

```
#define TMU_IP_MONITORING_SITE_0
```

Site masks. Use these masks wherever you want to select the site.

Implements : TMU_IP_MONITORING_SITE_BITMASK_Class

Definition at line 111 of file Tmu_Ip_Types.h.

6.2.3.3 TMU_IP_INTERRUPT_IMMEDIATE_HIGH_TEMP

```
#define TMU_IP_INTERRUPT_IMMEDIATE_HIGH_TEMP
```

The temperature threshold interrupt source masks.

These defines contains all TMU interrupt sources to be enabled or disabled.

Implements : TMU_IP_INTERRUPT_SOURCE_BITMASK_Class The immediate high temperature threshold interrupt source

Definition at line 123 of file Tmu_Ip_Types.h.

6.2.3.4 TMU_IP_INTERRUPT_AVERAGE_HIGH_TEMP

```
#define TMU_IP_INTERRUPT_AVERAGE_HIGH_TEMP
```

The average high temperature threshold interrupt source

Definition at line 124 of file Tmu_Ip_Types.h.

6.2.3.5 TMU_IP_INTERRUPT_AVERAGE_HIGH_TEMP_CRITICAL

```
#define TMU_IP_INTERRUPT_AVERAGE_HIGH_TEMP_CRITICAL
```

The average high temperature critical threshold interrupt source

Definition at line 125 of file Tmu_Ip_Types.h.

6.2.3.6 TMU_IP_INTERRUPT_IMMEDIATE_LOW_TEMP

```
#define TMU_IP_INTERRUPT_IMMEDIATE_LOW_TEMP
```

The immediate low temperature threshold interrupt source

Definition at line 126 of file Tmu_Ip_Types.h.

6.2.3.7 TMU_IP_INTERRUPT_AVERAGE_LOW_TEMP

```
#define TMU_IP_INTERRUPT_AVERAGE_LOW_TEMP
```

The average low temperature threshold interrupt source

Definition at line 127 of file Tmu_Ip_Types.h.

6.2.3.8 TMU_IP_INTERRUPT_AVERAGE_LOW_TEMP_CRITICAL

```
#define TMU_IP_INTERRUPT_AVERAGE_LOW_TEMP_CRITICAL
```

The average low temperature critical threshold interrupt source

Definition at line 128 of file Tmu_Ip_Types.h.

6.2.3.9 TMU_IP_INTERRUPT_RISING_TEMP_RATE_CRITICAL

```
#define TMU_IP_INTERRUPT_RISING_TEMP_RATE_CRITICAL
```

The rising temperature rate critical threshold interrupt source

Definition at line 129 of file Tmu_Ip_Types.h.

6.2.3.10 TMU_IP_INTERRUPT_FALLING_TEMP_RATE_CRITICAL

```
#define TMU_IP_INTERRUPT_FALLING_TEMP_RATE_CRITICAL
```

The falling temperature rate critical threshold interrupt source

Definition at line 130 of file Tmu_Ip_Types.h.

6.2.3.11 TMU_IP_STATUS_TIME_INTERVAL_EXCEEDED

```
#define TMU_IP_STATUS_TIME_INTERVAL_EXCEEDED
```

The monitoring status flag masks.

Monitoring and calibration status during operation.

Implements : TMU_IP_STATUS_FLAG_BITMASK_Class The monitoring interval exceeded flag

Definition at line 142 of file Tmu_Ip_Types.h.

6.2.3.12 TMU_IP_STATUS_OUT_OF_RANGE_LOW_TEMP

```
#define TMU_IP_STATUS_OUT_OF_RANGE_LOW_TEMP
```

The out-of-range low temperature measurement

Definition at line 143 of file Tmu_Ip_Types.h.

6.2.3.13 TMU_IP_STATUS_OUT_OF_RANGE_HIGH_TEMP

```
#define TMU_IP_STATUS_OUT_OF_RANGE_HIGH_TEMP
```

The out-of-range high temperature measurement

Definition at line 144 of file Tmu_Ip_Types.h.

6.2.4 Enum Reference

6.2.4.1 Tmu_Ip_AverageLowPassFilterType

```
enum Tmu_Ip_AverageLowPassFilterType
```

Average low pass filter setting.

This is used to set the average low pass filter The average temperature is calculated using this formula: ALPF x Current_Temp + (1 - ALPF) x Average_Temp

Enumerator

TMU_IP_AVERAGE_LOW_PASS_FILTER_1	Average low pass filter 1.0
TMU_IP_AVERAGE_LOW_PASS_FILTER_0_5	Average low pass filter 0.5
TMU_IP_AVERAGE_LOW_PASS_FILTER_0_25	Average low pass filter 0.25
TMU_IP_AVERAGE_LOW_PASS_FILTER_0_125	Average low pass filter 0.125

Definition at line 158 of file Tmu_Ip_Types.h.

6.2.4.2 Tmu_Ip_TempMonitoringIntervalType

```
enum Tmu_Ip_TempMonitoringIntervalType
```

Temperature monitoring interval setting.

This enum is used to set temperature monitoring interval in seconds. The time interval depends on TMU clock frequency and is determined using this formula: [2 ^ (23 + TMI)] x TMU clock period in seconds, where TMI is Temperature Monitoring Interval

Enumerator

TMU_IP_TEMP_MONITORING_INTERVAL_↩ CONTINUOUS	Disable temperature monitoring interval in seconds, monitoring is continuous

Definition at line 175 of file Tmu_Ip_Types.h.

6.2.4.3 Tmu_Ip_ReportTempType

```
enum Tmu_Ip_ReportTempType
```

Report temperature settings.

This is used to choose immediate temperature report or average report.

Enumerator

TMU_IP_REPORT_IMMEDIATE_TEMPERATURE	Report immediate temperature
TMU_IP_REPORT_AVERAGE_TEMPERATURE	Report average temperature

Definition at line 201 of file Tmu_Ip_Types.h.

6.2.4.4 Tmu_Ip_ThresholdType

```
enum Tmu_Ip_ThresholdType
```

Temperature threshold settings.

This is used to select type of threshold value.

Enumerator

TMU_IP_THRESHOLD_HIGH_TEMP_IMME↩ DIATE	The high temperature immediate threshold
TMU_IP_THRESHOLD_HIGH_TEMP_AVER↩ AGE	The high temperature average threshold
TMU_IP_THRESHOLD_HIGH_TEMP_AVER↩ AGE_CRITICAL	The high temperature average critical threshold
TMU_IP_THRESHOLD_LOW_TEMP_IMMED↩ IATE	The low temperature immediate threshold

Enumerator

TMU_IP_THRESHOLD_LOW_TEMP_AVERAGE	The low temperature average threshold
TMU_IP_THRESHOLD_LOW_TEMP_AVERAGE_CRITICAL	The low temperature average critical threshold
TMU_IP_THRESHOLD_RISING_TEMP_RATE_CRITICAL	The rising temperature difference between two measurements
TMU_IP_THRESHOLD_FALLING_TEMP_RATE_CRITICAL	The falling temperature difference between two measurements

Definition at line 213 of file Tmu_Ip_Types.h.

6.2.4.5 Tmu_Ip_ModeType

enum Tmu_Ip_ModeType

Working mode of the TMU peripheral. Values correspond to those in the MODE field of of the TMR register.

Definition at line 241 of file Tmu_Ip_Types.h.

6.2.4.6 Tmu_Ip_StatusType

enum Tmu_Ip_StatusType

Defines the Tmu status values.

Enumerator

TMU_IP_STATUS_SUCCESS	Function completed successfully
TMU_IP_STATUS_ERROR	Function didn't complete successfully
TMU_IP_STATUS_TIMEOUT	Function timed out
TMU_IP_STATUS_BUSY	Function busy

Definition at line 251 of file Tmu_Ip_Types.h.

6.2.5 Function Reference

6.2.5.1 Tmu_Ip_Init()

```
Tmu_Ip_StatusType Tmu_Ip_Init (
            const uint32 Instance,
```

```
            const Tmu_Ip_ConfigType *const ConfigPtr )
```

This function initializes ...

Parameters

in	*Instance*	The number of the TMU instance used.
in	*ConfigPtr*	Pointer to the user's configuration structure

Returns

status:

- TMU_IP_STATUS_SUCCESS: SAR ready to receive command
- TMU_IP_STATUS_TIMEOUT: SAR not ready to receive command

6.2.5.2 Tmu_Ip_Deinit()

```
void Tmu_Ip_Deinit (
            const uint32 Instance )
```

This function shall De-Init the TMU for all registers to default value.

Parameters

in	*Instance*	The number of the TMU instance used

6.2.5.3 Tmu_Ip_StartMonitoring()

```
void Tmu_Ip_StartMonitoring (
            const uint32 Instance )
```

This function starts monitoring the temperature on sites previously selected with Tmu_Ip_Init or Tmu_Ip_↩ EnableMonitoringSite. This function should be called after TMU initialization is done.

Parameters

in	*Instance*	The number of the TMU instance used

6.2.5.4 Tmu_Ip_StopMonitoring()

```
void Tmu_Ip_StopMonitoring (
             const uint32 Instance )
```

This function stop monitoring the temperature on sites.

Parameters

in	*Instance*	The number of the TMU instance used

6.2.5.5 Tmu_Ip_EnableMonitoringSite()

```
void Tmu_Ip_EnableMonitoringSite (
             const uint32 Instance,
             const uint8 TmuSiteMask )
```

This function shall enable the site which will be monitored. The site mask input value can be set using TMU_IP↩ _MONITORING_SITE_ defines.

Parameters

in	*Instance*	The number of the TMU instance used
in	*TmuSiteMask*	The masked value of site will be enabled

6.2.5.6 Tmu_Ip_DisableMonitoringSite()

```
void Tmu_Ip_DisableMonitoringSite (
             const uint32 Instance,
             const uint8 TmuSiteMask )
```

This function shall disable the selected site from monitoring. The site mask input value can be set using TMU_I↩ P_MONITORING_SITE_ defines.

Parameters

in	*Instance*	The number of the TMU instance used
in	*TmuSiteMask*	The masked value of site will be disabled

6.2.5.7 Tmu_Ip_SetTempThreshold()

```
void Tmu_Ip_SetTempThreshold (
             const uint32 Instance,
             const Tmu_Ip_ThresholdType ThresholdType,
             const uint16 ThresholdValue )
```

This function shall set the temperature threshold.

Parameters

in	*Instance*	The number of the TMU instance used
in	*ThresholdType*	Threshold type
in	*ThresholdValue*	Threshold value

6.2.5.8 Tmu_Ip_GetTempMinMaxLevel()

```
uint16 Tmu_Ip_GetTempMinMaxLevel (
             const uint32 Instance,
             const Tmu_Ip_TempLevelType TempLevelType,
             const boolean ClearValidFlag )
```

This function reads the minimum or maximum temperature recorded at one site. If the temperature is invalid, it will return TMU_IP_INVALID_TEMPERATURE value. The return value is Q9.1 format: 9 bits for highest temperature recorded and 1 bit for 0.5 degrees resolution. User can clear the current temperature value by setting ClearValidFlag parameter.

Parameters

in	*Instance*	The number of the TMU instance used
in	*TempLevelType*	Temperature level to be retrieved: min or max
in	*ClearValidFlag*	Allow clear the current temperature value

Returns

The highest temperature recorded

- For example:
  - if return value is 1100011100, the actual temperature is 398K
  - if return value is 1100011101, the actual temperature is 398.5K

uint16: Minimum or Maximum measured temperature value.

6.2.5.9 Tmu_Ip_GetTempMaxRate()

```
uint8 Tmu_Ip_GetTempMaxRate (
            const uint32 Instance,
            const Tmu_Ip_TempRateType TempRateType,
            const boolean ClearValidFlag )
```

Retrieves the maximum rate the temperature has risen or fallen, and clears that rate if ClearValidFlag is TRUE. If the temperature is invalid (cleared), it will return TMU_IP_INVALID_TEMPERATURE_RATE value.

Parameters

in	*Instance*	The number of the TMU instance used
in	*TempRateType*	Specifies whether to retrieve the rising or falling rate.
in	*ClearValidFlag*	TRUE to clear the rate.

Returns

The rising temperature rate

uint8: Minimum or Maximum measured temperature rate value.

6.2.5.10 Tmu_Ip_GetTemp()

```
void Tmu_Ip_GetTemp (
            const uint32 Instance,
            const uint8 TmuMonitoringSite,
            const Tmu_Ip_ReportTempType TmuTempType,
            uint16 *const TmuTempVal )
```

This function shall read the last temperature at the selected site If the temperature is invalid, it will return TMU_↩ IP_INVALID_TEMPERATURE value. The return value is Q9.1 format : 9 bits for highest temperature recorded and 1 bit for 0.5 degrees resolution.

Parameters

in	*Instance*	The number of the TMU instance used
in	*TmuMonitoringSite*	The monitored site index (0, 1, 2)
in	*TmuTempType*	The type of output temperature reported
out	*TmuTempVal*	The measured temperature value.

6.2.5.11 Tmu_Ip_EnableNotifications()

```
void Tmu_Ip_EnableNotifications (
```

```
            const uint32 Instance,
            const uint32 NotificationsMask )
```

This function enables the notifications for the selected types of thresholds. The interrupt mask input value can be set using TMU_IP_INTERRUPT_ defines.

Parameters

in	*Instance*	The number of the TMU instance used
in	*NotificationsMask*	The masked value of notifications to enable • For example: – With mask = TMU_IP_INTERRUPT_IMMEDIATE_HIGH_TEMP to enable immediate high temperature threshold interrupt(IHTTIE). – With mask = (TMU_IP_INTERRUPT_IMMEDIATE_HIGH_TEMP | TMU_IP_INTERRUPT_AVERAGE_HIGH_TEMP) to enable immediate high temperature threshold exceeded(IHTTIE) and average high temperature threshold exceeded(AHTTIE).

6.2.5.12 Tmu_Ip_DisableNotifications()

```
void Tmu_Ip_DisableNotifications (
            const uint32 Instance,
            const uint32 NotificationsMask )
```

This function disables the notifications for the selected types of thresholds. The interrupt mask input value can be set using TMU_IP_INTERRUPT_ defines.

Parameters

in	*Instance*	The number of the TMU instance used
in	*NotificationsMask*	The masked value of notifications to disable • For example: – With mask = TMU_IP_INTERRUPT_IMMEDIATE_HIGH_TEMP to disable immediate high temperature threshold interrupt(IHTTIE). – With mask = (TMU_IP_INTERRUPT_IMMEDIATE_HIGH_TEMP | TMU_IP_INTERRUPT_AVERAGE_HIGH_TEMP) to disable immediate high temperature threshold exceeded(IHTTIE) and average high temperature threshold exceeded(AHTTIE).

6.2.5.13 Tmu_Ip_SetMeasurementInterval()

```
void Tmu_Ip_SetMeasurementInterval (
            const uint32 Instance,
            const Tmu_Ip_TempMonitoringIntervalType TmuMonitoringInterval )
```

This function shall set the interval for the temperature monitoring. It should only be changed when monitoring is disabled.

Parameters

in	*Instance*	The number of the TMU instance used
in	*TmuMonitoringInterval*	The measurement interval

6.2.5.14 Tmu_Ip_GetMode()

```
Tmu_Ip_ModeType Tmu_Ip_GetMode (
            const uint32 Instance )
```

Retrieves the current mode. (TMU_IP_MODE_MONITORING after a call to Tmu_Ip_StartMonitoring, otherwise TMU_IP_MODE_IDLE_LOW_POWER)

Parameters

in	*Instance*	The number of the TMU instance used

Returns

Tmu_Ip_ModeType: Mode, low power or monitoring.

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, AltiVec, C–5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C–Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.